

The Arrow C data interface

Rationale

Apache Arrow is designed to be a universal in-memory format for the representation of tabular (“columnar”) data. However, some projects may face a difficult choice between either depending on a fast-evolving project such as the Arrow C++ library, or having to reimplement adapters for data interchange, which may require significant, redundant development effort.

The Arrow C data interface defines a very small, stable set of C definitions that can be easily *copied* in any project’s source code and used for columnar data interchange in the Arrow format. For non-C/C++ languages and runtimes, it should be almost as easy to translate the C definitions into the corresponding C FFI declarations.

Applications and libraries can therefore work with Arrow memory without necessarily using Arrow libraries or reinventing the wheel. Developers can choose between tight integration with the Arrow *software project* (benefiting from the growing array of facilities exposed by e.g. the C++ or Java implementations of Apache Arrow, but with the cost of a dependency) or minimal integration with the Arrow *format* only.

Goals

- Expose an ABI-stable interface.
- Make it easy for third-party projects to implement support for (including partial support where sufficient), with little initial investment.
- Allow zero-copy sharing of Arrow data between independent runtimes and components running in the same process.
- Match the Arrow array concepts closely to avoid the development of yet another marshalling layer.
- Avoid the need for one-to-one adaptation layers such as the limited JType-based bridge between Java and Python.
- Enable integration without an explicit dependency (either at compile-time or runtime) on the Arrow software project.



Ask AI

Ideally, the Arrow C data interface can become a low-level *lingua franca* for sharing columnar data at runtime and establish Arrow as the universal building block in the columnar processing ecosystem.

Non-goals

- Expose a C API mimicking operations available in higher-level runtimes (such as C++, Java...).
- Data sharing between distinct processes or storage persistence.

Comparison with the Arrow IPC format

Pros of the C data interface vs. the IPC format:

- No dependency on Flatbuffers.
- No buffer reassembly (data is already exposed in logical Arrow format).
- Zero-copy by design.
- Easy to reimplement from scratch.
- Minimal C definition that can be easily copied into other codebases.
- Resource lifetime management through a custom release callback.

Pros of the IPC format vs. the data interface:

- Works across processes and machines.
- Allows data storage and persistence.
- Being a streamable format, the IPC format has room for composing more features (such as integrity checks, compression...).
- Does not require explicit C data access.

Data type description – format strings

A data type is described using a format string. The format string only encodes information about the top-level type; for nested type, child types are described separately. Also, metadata is encoded in a separate string.

The format strings are designed to be easily parsable, even from a language such as C. The most common primitive formats have one-character format strings:

Format string	Arrow data type	Notes
<code>n</code>	null	
<code>b</code>	boolean	
<code>c</code>	int8	
<code>C</code>	uint8	
<code>s</code>	int16	
<code>S</code>	uint16	
<code>i</code>	int32	
<code>I</code>	uint32	
<code>l</code>	int64	
<code>L</code>	uint64	
<code>e</code>	float16	
<code>f</code>	float32	
<code>g</code>	float64	

Format string	Arrow data type	Notes
<code>z</code>	binary	
<code>Z</code>	large binary	
<code>vz</code>	binary view	
<code>u</code>	utf-8 string	
<code>U</code>	large utf-8 string	

Format string	Arrow data type	Notes
<code>vu</code>	utf-8 view	
<code>d:19,10</code>	decimal128 [precision 19, scale 10]	
<code>d:19,10,NNN</code>	decimal bitwidth = NNN [precision 19, scale 10]	
<code>w:42</code>	fixed-width binary [42 bytes]	

Temporal types have multi-character format strings starting with `t`:

Format string	Arrow data type	Notes
<code>tdD</code>	date32 [days]	
<code>tdm</code>	date64 [milliseconds]	
<code>tts</code>	time32 [seconds]	
<code>ttm</code>	time32 [milliseconds]	
<code>ttu</code>	time64 [microseconds]	
<code>ttn</code>	time64 [nanoseconds]	
<code>tss: ...</code>	timestamp [seconds] with timezone "..."	(1)
<code>tsm: ...</code>	timestamp [milliseconds] with timezone "..."	(1)
<code>tsu: ...</code>	timestamp [microseconds] with timezone "..."	(1)
<code>tsn: ...</code>	timestamp [nanoseconds] with timezone "..."	(1)
<code>tDs</code>	duration [seconds]	
<code>tDm</code>	duration [milliseconds]	
<code>tDu</code>	duration [microseconds]	
<code>tDn</code>	duration [nanoseconds]	
<code>tiM</code>	interval [months]	

Format string	Arrow data type	Notes
<code>tiD</code>	interval [days, time]	
<code>tin</code>	interval [month, day, nanoseconds]	

Dictionary-encoded types do not have a specific format string. Instead, the format string of the base array represents the dictionary index type, and the value type can be read from the dependent dictionary array (see below “Dictionary-encoded arrays”).

Nested types have multiple-character format strings starting with `+`. The names and types of child fields are read from the child arrays.

Format string	Arrow data type	Notes
<code>+l</code>	list	
<code>+L</code>	large list	
<code>+vl</code>	list-view	
<code>+vL</code>	large list-view	
<code>+w:123</code>	fixed-sized list [123 items]	
<code>+s</code>	struct	
<code>+m</code>	map	(2)
<code>+ud:I,J, ...</code>	dense union with type ids I,J...	
<code>+us:I,J, ...</code>	sparse union with type ids I,J...	
<code>+r</code>	run-end encoded	(3)

Notes:

1. The timezone string is appended as-is after the colon character `:`, without any quotes. If the timezone is empty, the colon `:` must still be included.
2. As specified in the Arrow columnar format, the map type has a single child type named `entries`, itself a 2-child struct type of `(key, value)`.

3. As specified in the Arrow columnar format, the run-end encoded type has two children where the first is the (integral) `run_ends` and the second is the `values`.

Examples

- A dictionary-encoded `decimal128(precision = 12, scale = 5)` array with `int16` indices has format string `s`, and its dependent dictionary array has format string `d:12,5`.
- A `list<uint64>` array has format string `+l`, and its single child has format string `L`.
- A `large_list_view<uint64>` array has format string `+vL`, and its single child has format string `L`.
- A `struct<ints: int32, floats: float32>` has format string `+s`; its two children have names `ints` and `floats`, and format strings `i` and `f` respectively.
- A `map<string, float64>` array has format string `+m`; its single child has name `entries` and format string `+s`; its two grandchildren have names `key` and `value`, and format strings `u` and `g` respectively.
- A `sparse_union<ints: int32, floats: float32>` with type ids `4, 5` has format string `+us:4,5`; its two children have names `ints` and `floats`, and format strings `i` and `f` respectively.
- A `run_end_encoded<int32, float32>` has format string `+r`; its two children have names `run_ends` and `values`, and format strings `i` and `f` respectively.

Structure definitions

The following free-standing definitions are enough to support the Arrow C data interface in your project. Like the rest of the Arrow project, they are available under the Apache License 2.0.

```

#ifndef ARROW_C_DATA_INTERFACE
#define ARROW_C_DATA_INTERFACE

#define ARROW_FLAG_DICTIONARY_ORDERED 1
#define ARROW_FLAG_NULLABLE 2
#define ARROW_FLAG_MAP_KEYS_SORTED 4

struct ArrowSchema {
    // Array type description
    const char* format;
    const char* name;
    const char* metadata;
    int64_t flags;
    int64_t n_children;
    struct ArrowSchema** children;
    struct ArrowSchema* dictionary;

    // Release callback
    void (*release)(struct ArrowSchema*);
    // Opaque producer-specific data
    void* private_data;
};

struct ArrowArray {
    // Array data description
    int64_t length;
    int64_t null_count;
    int64_t offset;
    int64_t n_buffers;
    int64_t n_children;
    const void** buffers;
    struct ArrowArray** children;
    struct ArrowArray* dictionary;

    // Release callback
    void (*release)(struct ArrowArray*);
    // Opaque producer-specific data
    void* private_data;
};

#endif // ARROW_C_DATA_INTERFACE

```

Note

The canonical guard `ARROW_C_DATA_INTERFACE` is meant to avoid duplicate definitions if two projects copy the C data interface definitions in their own headers, and a third-party project includes from these two projects. It is therefore important that this guard is kept exactly as-is when these definitions are copied.

The ArrowSchema structure

The `ArrowSchema` structure describes the type and metadata of an exported array or record batch. It has the following fields:

`const char *ArrowSchema.format`

Mandatory. A null-terminated, UTF8-encoded string describing the data type. If the data type is nested, child types are not encoded here but in the `ArrowSchema.children` structures.

Consumers MAY decide not to support all data types, but they should document this limitation.

`const char *ArrowSchema.name`

Optional. A null-terminated, UTF8-encoded string of the field or array name. This is mainly used to reconstruct child fields of nested types.

Producers MAY decide not to provide this information, and consumers MAY decide to ignore it. If omitted, MAY be NULL or an empty string.

`const char *ArrowSchema.metadata`

Optional. A binary string describing the type's metadata. If the data type is nested, child types are not encoded here but in the `ArrowSchema.children` structures.

This string is not null-terminated but follows a specific format:

```
int32: number of key/value pairs (noted N below)
int32: byte length of key 0
key 0 (not null-terminated)
int32: byte length of value 0
value 0 (not null-terminated)
...
int32: byte length of key N - 1
key N - 1 (not null-terminated)
int32: byte length of value N - 1
value N - 1 (not null-terminated)
```

Integers are stored in native endianness. For example, the metadata `[('key1', 'value1')]` is encoded on a little-endian machine as:


```
\x01\x00\x00\x00\x04\x00\x00\x00key1\x06\x00\x00\x00value1
```

On a big-endian machine, the same example would be encoded as:

```
\x00\x00\x00\x01\x00\x00\x00\x04key1\x00\x00\x00\x06value1
```

If omitted, this field MUST be NULL (not an empty string).

Consumers MAY choose to ignore this information.

int64_t ArrowSchema.flags

Optional. A bitfield of flags enriching the type description. Its value is computed by OR'ing together the flag values. The following flags are available:

- **ARROW_FLAG_NULLABLE**: whether this field is semantically nullable (regardless of whether it actually has null values).
- **ARROW_FLAG_DICTIONARY_ORDERED**: for dictionary-encoded types, whether the ordering of dictionary indices is semantically meaningful.
- **ARROW_FLAG_MAP_KEYS_SORTED**: for map types, whether the keys within each map value are sorted.

If omitted, MUST be 0.

Consumers MAY choose to ignore some or all of the flags. Even then, they SHOULD keep this value around so as to propagate its information to their own consumers.

int64_t ArrowSchema.n_children

Mandatory. The number of children this type has.

ArrowSchema **ArrowSchema.children

Optional. A C array of pointers to each child type of this type. There must be

ArrowSchema.n_children pointers.

MAY be NULL only if **ArrowSchema.n_children** is 0.

ArrowSchema *ArrowSchema.dictionary

Optional. A pointer to the type of dictionary values.

MUST be present if the ArrowSchema represents a dictionary-encoded type. MUST be NULL otherwise.

void (*ArrowSchema.release)(struct ArrowSchema*)

Mandatory. A pointer to a producer-provided release callback.

See below for memory management and release callback semantics.

void *ArrowSchema.private_data

Optional. An opaque pointer to producer-provided private data.

Consumers MUST not process this member. Lifetime of this member is handled by the producer, and especially by the release callback.

The ArrowArray structure

The `ArrowArray` describes the data of an exported array or record batch. For the `ArrowArray` structure to be interpreted type, the array type or record batch schema must already be known. This is either done by convention – for example a producer API that always produces the same data type – or by passing a `ArrowSchema` on the side.

It has the following fields:

int64_t ArrowArray.length

Mandatory. The logical length of the array (i.e. its number of items).

int64_t ArrowArray.null_count

Mandatory. The number of null items in the array. MAY be -1 if not yet computed.

int64_t ArrowArray.offset

Mandatory. The logical offset inside the array (i.e. the number of items from the physical start of the buffers). MUST be 0 or positive.

Producers MAY specify that they will only produce 0-offset arrays to ease implementation of consumer code. Consumers MAY decide not to support non-0-offset arrays, but they should document this limitation.

int64_t ArrowArray.n_buffers

Mandatory. The number of physical buffers backing this array. The number of buffers is a function of the data type, as described in the [Columnar format specification](#), except for the binary or utf-8 view type, which has one additional buffer compared to the Columnar format specification (see [Binary view arrays](#)).

Buffers of children arrays are not included.

const void **ArrowArray.buffers

Mandatory. A C array of pointers to the start of each physical buffer backing this array. Each `void*` pointer is the physical start of a contiguous buffer. There must be

`ArrowArray.n_buffers` pointers.

The producer MUST ensure that each contiguous buffer is large enough to represent *length* + *offset* values encoded according to the [Columnar format specification](#).

It is recommended, but not required, that the memory addresses of the buffers be aligned at least according to the type of primitive data that they contain. Consumers MAY decide not to support unaligned memory.

The buffer pointers MAY be null only in two situations:

1. for the null bitmap buffer, if `ArrowArray.null_count` is 0;
2. for any buffer (including variadic buffers), if the size in bytes of the corresponding buffer would be 0.

Buffers of children arrays are not included.

int64_t ArrowArray.n_children

Mandatory. The number of children this array has. The number of children is a function of the data type, as described in the [Columnar format specification](#).

ArrowArray **ArrowArray.children

Optional. A C array of pointers to each child array of this array. There must be `ArrowArray.n_children` pointers.

MAY be NULL only if `ArrowArray.n_children` is 0.

ArrowArray *ArrowArray.dictionary

Optional. A pointer to the underlying array of dictionary values.

MUST be present if the ArrowArray represents a dictionary-encoded array. MUST be NULL otherwise.

void (*ArrowArray.release)(struct ArrowArray*)

Mandatory. A pointer to a producer-provided release callback.

See below for memory management and release callback semantics.

void *ArrowArray.private_data

Optional. An opaque pointer to producer-provided private data.

Consumers MUST not process this member. Lifetime of this member is handled by the producer, and especially by the release callback.

Dictionary-encoded arrays

For dictionary-encoded arrays, the `ArrowSchema.format` string encodes the *index* type. The dictionary *value* type can be read from the `ArrowSchema.dictionary` structure.

The same holds for `ArrowArray` structure: while the parent structure points to the index data, the `ArrowArray.dictionary` points to the dictionary values array.

Extension arrays

For extension arrays, the `ArrowSchema.format` string encodes the *storage* type. Information about the extension type is encoded in the `ArrowSchema.metadata` string, similarly to the [IPC format](#). Specifically, the metadata key `ARROW:extension:name` encodes the extension type name, and the metadata key `ARROW:extension:metadata` encodes the implementation-specific serialization of the extension type (for parameterized extension types).

The `ArrowArray` structure exported from an extension array simply points to the storage data of the extension array.

Binary view arrays

For binary or utf-8 view arrays, an extra buffer is appended which stores the lengths of each variadic data buffer as `int64_t`. This buffer is necessary since these buffer lengths are not trivially extractable from other data in an array of binary or utf-8 view type.

Semantics

Memory management

The `ArrowSchema` and `ArrowArray` structures follow the same conventions for memory management. The term “*base structure*” below refers to the `ArrowSchema` or `ArrowArray` that is passed between producer and consumer – not any child structure thereof.

Member allocation

It is intended for the base structure to be stack- or heap-allocated by the consumer. In this case, the producer API should take a pointer to the consumer-allocated structure.

However, any data pointed to by the struct **MUST** be allocated and maintained by the producer. This includes the format and metadata strings, the arrays of buffer and children pointers, etc.

Therefore, the consumer **MUST** not try to interfere with the producer’s handling of these members’ lifetime. The only way the consumer influences data lifetime is by calling the base structure’s

`release` callback.

Released structure

A released structure is indicated by setting its `release` callback to NULL. Before reading and interpreting a structure's data, consumers SHOULD check for a NULL release callback and treat it accordingly (probably by erroring out).

Release callback semantics – for consumers

Consumers MUST call a base structure's release callback when they won't be using it anymore, but they MUST not call any of its children's release callbacks (including the optional dictionary). The producer is responsible for releasing the children.

In any case, a consumer MUST not try to access the base structure anymore after calling its release callback – including any associated data such as its children.

Release callback semantics – for producers

If producers need additional information for lifetime handling (for example, a C++ producer may want to use `shared_ptr` for array and buffer lifetime), they MUST use the `private_data` member to locate the required bookkeeping information.

The release callback MUST not assume that the structure will be located at the same memory location as when it was originally produced. The consumer is free to move the structure around (see "Moving an array").

The release callback MUST walk all children structures (including the optional dictionary) and call their own release callbacks.

The release callback MUST free any data area directly owned by the structure (such as the buffers and children members).

The release callback MUST mark the structure as released, by setting its `release` member to NULL.

Below is a good starting point for implementing a release callback, where the TODO area must be filled with producer-specific deallocation code:

```

static void ReleaseExportedArray(struct ArrowArray* array) {
    // This should not be called on already released array
    assert(array->release != NULL);

    // Release children
    for (int64_t i = 0; i < array->n_children; ++i) {
        struct ArrowArray* child = array->children[i];
        if (child->release != NULL) {
            child->release(child);
            assert(child->release == NULL);
        }
    }

    // Release dictionary
    struct ArrowArray* dict = array->dictionary;
    if (dict != NULL && dict->release != NULL) {
        dict->release(dict);
        assert(dict->release == NULL);
    }

    // TODO here: release and/or deallocate all data directly owned by
    // the ArrowArray struct, such as the private_data.

    // Mark array released
    array->release = NULL;
}

```

Moving an array

The consumer can *move* the `ArrowArray` structure by bitwise copying or shallow member-wise copying. Then it **MUST** mark the source structure released (see “released structure” above for how to do it) but *without* calling the release callback. This ensures that only one live copy of the struct is active at any given time and that lifetime is correctly communicated to the producer.

As usual, the release callback will be called on the destination structure when it is not needed anymore.

Moving child arrays

It is also possible to move one or several child arrays, but the parent `ArrowArray` structure **MUST** be released immediately afterwards, as it won't point to valid child arrays anymore.

The main use case for this is to keep alive only a subset of child arrays (for example if you are only interested in certain columns of the data), while releasing the others.

Note

For moving to work correctly, the `ArrowArray` structure has to be trivially relocatable. Therefore, pointer members inside the `ArrowArray` structure (including `private_data`) MUST not point inside the structure itself. Also, external pointers to the structure MUST not be separately stored by the producer. Instead, the producer MUST use the `private_data` member so as to remember any necessary bookkeeping information.

Record batches

A record batch can be trivially considered as an equivalent struct array. In this case the metadata of the top-level `ArrowSchema` can be used for the schema-level metadata of the record batch.

Mutability

Both the producer and the consumer SHOULD consider the exported data (that is, the data reachable through the `buffers` member of `ArrowArray`) to be immutable, as either party could otherwise see inconsistent data while the other is mutating it.

Example use case

A C++ database engine wants to provide the option to deliver results in Arrow format, but without imposing themselves a dependency on the Arrow software libraries. With the Arrow C data interface, the engine can let the caller pass a pointer to a `ArrowArray` structure, and fill it with the next chunk of results.

It can do so without including the Arrow C++ headers or linking with the Arrow DLLs. Furthermore, the database engine's C API can benefit other runtimes and libraries that know about the Arrow C data interface, through e.g. a C FFI layer.

C producer examples

Exporting a simple `int32` array

Export a non-nullable `int32` type with empty metadata. In this case, all `ArrowSchema` members point to statically-allocated data, so the release callback is trivial.

```
static void release_int32_type(struct ArrowSchema* schema) {
    // Mark released
    schema->release = NULL;
}

void export_int32_type(struct ArrowSchema* schema) {
    *schema = (struct ArrowSchema) {
        // Type description
        .format = "i",
        .name = "",
        .metadata = NULL,
        .flags = 0,
        .n_children = 0,
        .children = NULL,
        .dictionary = NULL,
        // Bookkeeping
        .release = &release_int32_type
    };
}
```

Export a C-malloc()ed array of the same type as a Arrow array, transferring ownership to the consumer through the release callback:

```

static void release_int32_array(struct ArrowArray* array) {
    assert(array->n_buffers == 2);
    // Free the buffers and the buffers array
    free((void *) array->buffers[1]);
    free(array->buffers);
    // Mark released
    array->release = NULL;
}

void export_int32_array(const int32_t* data, int64_t nitems,
                      struct ArrowArray* array) {
    // Initialize primitive fields
    *array = (struct ArrowArray) {
        // Data description
        .length = nitems,
        .offset = 0,
        .null_count = 0,
        .n_buffers = 2,
        .n_children = 0,
        .children = NULL,
        .dictionary = NULL,
        // Bookkeeping
        .release = &release_int32_array
    };
    // Allocate list of buffers
    array->buffers = (const void**) malloc(sizeof(void*) * array->n_buffers);
    assert(array->buffers != NULL);
    array->buffers[0] = NULL; // no nulls, null bitmap can be omitted
    array->buffers[1] = data;
}

```

Exporting a `struct<float32, utf8>` array

Export the array type as a `ArrowSchema` with C-malloc()ed children:

```

static void release_mallocated_type(struct ArrowSchema* schema) {
    int i;
    for (i = 0; i < schema->n_children; ++i) {
        struct ArrowSchema* child = schema->children[i];
        if (child->release != NULL) {
            child->release(child);
        }
        free(child);
    }
    free(schema->children);
    // Mark released
    schema->release = NULL;
}

void export_float32_utf8_type(struct ArrowSchema* schema) {
    struct ArrowSchema* child;

    //
    // Initialize parent type
    //
    *schema = (struct ArrowSchema) {
        // Type description
        .format = "+s",
        .name = "",
        .metadata = NULL,
        .flags = 0,
        .n_children = 2,
        .dictionary = NULL,
        // Bookkeeping
        .release = &release_mallocated_type
    };
    // Allocate list of children types
    schema->children = malloc(sizeof(struct ArrowSchema*) * schema->n_children);

    //
    // Initialize child type #0
    //
    child = schema->children[0] = malloc(sizeof(struct ArrowSchema));
    *child = (struct ArrowSchema) {
        // Type description
        .format = "f",
        .name = "floats",
        .metadata = NULL,
        .flags = ARROW_FLAG_NULLABLE,
        .n_children = 0,
        .dictionary = NULL,
        .children = NULL,
        // Bookkeeping
        .release = &release_mallocated_type
    };

    //
    // Initialize child type #1
    //
    child = schema->children[1] = malloc(sizeof(struct ArrowSchema));

```

```
*child = (struct ArrowSchema) {  
    // Type description  
    .format = "u",  
    .name = "strings",  
    .metadata = NULL,  
    .flags = ARROW_FLAG_NULLABLE,  
    .n_children = 0,  
    .dictionary = NULL,  
    .children = NULL,  
    // Bookkeeping  
    .release = &release_malloced_type  
};  
}
```

Export C-malloc()ed arrays in Arrow-compatible layout as an Arrow struct array, transferring ownership to the consumer:

```

static void release_mallocated_array(struct ArrowArray* array) {
    int i;
    // Free children
    for (i = 0; i < array->n_children; ++i) {
        struct ArrowArray* child = array->children[i];
        if (child->release != NULL) {
            child->release(child);
        }
        free(child);
    }
    free(array->children);
    // Free buffers
    for (i = 0; i < array->n_buffers; ++i) {
        free((void *) array->buffers[i]);
    }
    free(array->buffers);
    // Mark released
    array->release = NULL;
}

void export_float32_utf8_array(
    int64_t nitems,
    const uint8_t* float32_nulls, const float* float32_data,
    const uint8_t* utf8_nulls, const int32_t* utf8_offsets, const uint8_t* utf8_data,
    struct ArrowArray* array) {
    struct ArrowArray* child;

    //
    // Initialize parent array
    //
    *array = (struct ArrowArray) {
        // Data description
        .length = nitems,
        .offset = 0,
        .null_count = 0,
        .n_buffers = 1,
        .n_children = 2,
        .dictionary = NULL,
        // Bookkeeping
        .release = &release_mallocated_array
    };
    // Allocate list of parent buffers
    array->buffers = malloc(sizeof(void*) * array->n_buffers);
    array->buffers[0] = NULL; // no nulls, null bitmap can be omitted
    // Allocate list of children arrays
    array->children = malloc(sizeof(struct ArrowArray*) * array->n_children);

```

© Copyright 2016-2026 Apache Software Foundation. Apache Arrow, Arrow, Apache, the Apache logo, and the Apache Arrow project logo are either registered trademarks or trademarks of The Apache Software Foundation in the United States and other countries.

Created using [Sphinx](#) 9.1.0.

Built with the
[PyData](#)
[Sphinx](#)
[Theme](#) 0.19.0.